

INK MORROW 4.0 DOCUMENTATION LIBRARY

Operations & Recovery Handbook

Install, protect, update, diagnose, and restore Ink Morrow 4.0

For the owner-operator / September 2026

Operations & Recovery Handbook

This handbook is the practical companion for the person who owns the Ink Morrow installation. It explains not only which command to run, but what that command changes, how to recognize a healthy result, and how to get back to safety when something does not behave as expected.

Scope: Ink Morrow 4.0.0 beta, single-owner self-hosting, Windows/macOS/Linux, current Google Chrome.

Reading paths: New operators should read Chapters 1-4 and 7. Experienced operators can begin with the checklists. During an incident, start with Chapter 10 and resist the urge to improvise destructive database repairs.

The operational rule is simple: protect the whole data directory, change one thing at a time, and make the application prove that the result is healthy before writing resumes.

Contents

The installation in plain language

Before installation

Install and first start

Configuration without guesswork

Provider setup and model roles

Backup strategy: two different safety nets

Safe updates

Restore and transfer

Network access and HTTPS

Routine health and housekeeping

Incident triage

Recovery decision tree

Command reference

Operator checklists

01 The installation in plain language

Ink Morrow is one Node.js program. It serves the browser interface and its private API from the same address, stores structured information in one SQLite database, and stores images, audio, transfers, and safety backups beside it in a data directory. There is no Ink Morrow cloud account.

Part	What it does	What the operator protects
Application checkout	Program code and documentation	The reviewed commit or release tag
DATA_DIR	Home for the database and media	Copy it as one indivisible unit
SQLite database	Manuscripts, revisions, canon, settings, job records	Never edit it while Ink Morrow is running
Media directories	Art, audio, staged transfers, safety backups	Keep them paired with their database
Browser	Presents the application	Current Chrome is the tested client
AI provider	Optional paid drafting, memory, imagery, narration	Provider key, limits, compatible models

Ink Morrow binds to `127.0.0.1` by default. That address means "this computer only." Another device cannot reach it unless you deliberately configure a safe network path.

TESTED BOUNDARIES

Google Chrome is the only browser tested. OpenRouter is the only AI supplier tested. A current standards-respecting browser should work. Another OpenAI-compatible supplier may omit model discovery, image generation, narration, reasoning controls, or may not work at all.

02 Before installation

Install Node.js 22.5 or newer and Git. Choose two locations:

1. A checkout directory for the code. It can be replaced by a fresh clone.
2. A private data directory for the irreplaceable manuscripts and media. It must be included in backups.

Record these facts in a small operator note:

- checkout path;
- `DATA_DIR` and any `DB_PATH` override;
- installed commit (`git rev-parse HEAD`);
- Node version (`node --version`);
- listening host and port;
- public origin and reverse proxy, if any;
- where cold backups are stored; and
- who can access the provider account.

DO NOT REUSE 3.X STORAGE

Ink Morrow 4.0 is a clean data-contract break. It refuses a 3.x database before mutation. Keep the historical installation and its data intact. Do not rename a 3.x database and expect its identity to change.

03 Install and first start

Clone and install from the repository root:

```
git clone https://github.com/rthorman/ink-morrow.git
cd ink-morrow
npm ci
cd backend && npm ci
cd ../frontend && npm ci
cd ../e2e && npm ci
cd ..
```

Copy `backend/.env.example` to `backend/.env`, then set only values you understand. Start with loopback defaults. A minimal local installation normally needs no network-facing host override.

```
npm start
```

On first start, the terminal prints a one-time setup code. Open `http://localhost:3000` in Chrome, enter that code, and create the owner passphrase. The passphrase protects the private application and can also unlock locally saved provider credentials.

First-start acceptance check

- The terminal reports the exact local URL and no database refusal.
- The browser shows the Ink Morrow threshold, not a raw error or directory listing.
- Setup creates the owner and then stops accepting the one-time code.
- Lock, unlock, and refresh behave consistently.
- Library opens without an AI key.
- `DATA_DIR` contains the expected database and application-owned folders.

04 Configuration without guesswork

Environment values are read when the process starts. Editing `.env` does nothing until Ink Morrow restarts.

Setting	Purpose	Safe default
HOST	Interface on which the server listens	127.0.0.1
PORT	Local TCP port	3000
DATA_DIR	Complete application-data root	Explicit private path
DB_PATH	Advanced database-only override	Leave unset
ALLOWED_HOSTS	Host names accepted by the server	Local host names only
TRUST_PROXY	Trust proxy-supplied HTTPS information	Off unless a reviewed proxy is present
PUBLIC_ORIGIN	Address used for public snapshot links	HTTPS URL when sharing is enabled
OPENROUTER_API_KEY	Environment-provided provider credential	Dedicated, spending-limited key
CONTINUITY_MODEL	Server-owned Archivist model	A verified JSON-capable OpenRouter model

The configured continuity model is validated against OpenRouter before the server begins listening. A typo or unavailable explicit model causes startup to fail. This is deliberate: silently choosing a different model would make memory behavior and cost unpredictable.

CREDENTIALS IN `.ENV`

An environment key is convenient but is plain text on disk. Restrict file permissions. Prefer a dedicated provider key with a hard upstream spending limit. Never paste real keys into bug reports, screenshots, command history, or repository files.

05 Provider setup and model roles

Ink Morrow has three logical roles:

Role	Work	Important capability
Scribe	Prose drafting, direction, preparation	Long context and reliable text generation
Archivist	Chronicle memory and Codex impact summaries	Strict JSON/schema following
Narrator	Read-aloud and audiobook jobs	Supported audio output and voice

One model may fill several roles, but the roles remain separate configuration decisions. In particular, Chronicle must use the server-configured Archivist. A model selected in one browser tab must not silently replace it.

Before the first paid action, confirm the provider, model, data categories, references, operation count, and estimated cost. Estimates are guidance, not invoices. Provider-side hard limits are the actual financial boundary.

06 Backup strategy: two different safety nets

Use both portable and cold backups. They solve different problems.

Portable `.inkmorrow` archive

Create it in Gate. For a full-fidelity project backup, select visuals, audio, and working history. The archive is a versioned ZIP container with a manifest, ordinary JSON, and selected media. It is designed for reviewed transfer and restore.

It deliberately excludes owner credentials, sessions, provider secrets, saved paid consent, recovery suffixes, undo credentials, and local share capabilities. It is unencrypted; store it as sensitively as the manuscript.

Cold `DATA_DIR` copy

Stop Ink Morrow, then copy the complete data directory as one unit. This is the disaster-recovery image. It preserves local-only owner, session, vault, recovery, share, media, and database state.

Never combine a database from one date with media folders from another. Their references are a single consistency boundary.

Recommended cadence

Event	Portable archive	Cold copy
Normal writing	After meaningful sessions	Daily or automated snapshot
Before update	Required	Required with application stopped
Before full replace import	Ink Morrow also creates a safety archive	Recommended
Before storage move	Required	Required
After major recovery	Validate and create anew	Create after validation

07 Safe updates

1. Finish or cancel visible provider and publication jobs.
2. Create and download a full Gate backup.
3. Stop Ink Morrow.
4. Make a dated cold copy of the whole `DATA_DIR`.
5. Record current commit and Node version.
6. Fetch the reviewed change and install exact lockfile dependencies with `npm ci`.
7. Start once and read the complete startup result.
8. Verify Library counts, manuscript title, cast, Chronicle coverage, Codex facts, placed art, and Gate formats.
9. Only then resume authoring.

Earlier 4.0 beta databases from before the Ink Morrow naming change are adopted only after their 4.0 identity and migration checksums are proven. Before changing identity, startup writes a complete `*.pre-ink-morrow-v4.bak` SQLite snapshot beside the database and prints its location.

STOP ON IDENTITY ERRORS

Do not solve a family/version refusal by editing metadata, renaming files, or deleting the new database. Preserve the exact error and paths. Identity checks exist to prevent the wrong migrations from touching the wrong data.

08 Restore and transfer

Open Gate and preflight the archive before committing it. Preflight validates the archive family/version, declared files, hashes, paths, expansion limits, and collisions without changing the catalogue.

Collision choices are whole-entity choices: add, reuse identical data, keep, copy with new IDs, or replace. Ink Morrow does not attempt field-level world/character merges or page splicing. A copied manuscript is remapped as one dependency graph.

Replace everything is reserved for a deliberate full restore. Ink Morrow first creates a persistent safety archive of the current installation under the transfer backup directory. Download that safety archive before considering cleanup.

After restore, validate:

- manuscript hierarchy and page order;
- active and historical revisions;
- main/support/background cast tiers and character facts;
- world facts and events;
- continuity coverage and corrections;
- prepared page identity;
- placed and unplaced media;
- publication snapshots; and
- sanitized settings.

09 Network access and HTTPS

HTTP moves readable traffic. HTTPS encrypts traffic between a browser and the public front door and authenticates the site certificate. It matters because an Ink Morrow session, private prose, provider actions, and public capability links should not be exposed to other devices on the route.

For another device or public reading links, keep Ink Morrow on loopback and put a reviewed HTTPS reverse proxy in front:

```
HOST=127.0.0.1
PORT=3000
ALLOWED_HOSTS=ink.example.com
TRUST_PROXY=1
PUBLIC_ORIGIN=https://ink.example.com
```

The proxy must preserve `Host`, set `X-Forwarded-Proto: https`, forward authorization, and avoid logging or caching share capabilities. Public snapshot creation fails closed when the configured origin is insecure.

`ALLOW_INSECURE_LAN=1` is a temporary private-LAN escape hatch, not an Internet deployment plan.

CAPABILITY LINKS ARE KEYS

Anyone holding a live public snapshot URL can read that immutable snapshot. Do not put the token in analytics, logs, screenshots, or public messages. Revoke a link that may have escaped.

10 Routine health and housekeeping

Weekly:

- confirm a recent portable archive opens in preflight;
- confirm a recent cold backup exists outside the live disk;
- review provider spend at the provider, not only in Ink Morrow;
- review failed Chronicle entries and unfinished publication jobs;
- check free disk space, especially before image/audio work;
- inspect startup and application logs for repeated errors; and
- confirm the installed commit is the intended reviewed version.

Monthly or before a release:

- restore a backup into a separate empty `DATA_DIR` ;
- open representative DOCX, EPUB, PDF, and `.inkmorrow` outputs;
- revoke obsolete share links;
- apply supported Node/security updates in a tested branch; and
- document any known workaround.

11 Incident triage

First protect evidence: stop repeated clicks, note the exact time and manuscript, capture the complete visible error, record commit/configuration without secrets, and make a cold copy if corruption is suspected.

Symptom	First checks	Safe response
Server refuses to start	First error, model spelling, data identity, port use	Correct configuration; never delete data to silence the check
Chronicle says Memory failed	Click the error; inspect reason/model; compare heavy cast	Repair the specific page in Codex; keep Main Character perspective anchor
Repair repeatedly fails	JSON/schema capability, provider response, context pressure	Use verified Archivist; capture sanitized diagnostics; do not replay whole novel
Browser cannot connect	Server process, URL, host/port, firewall/proxy	Test <code>localhost</code> on server first, then each network layer
Another device behaves differently	Chrome version, viewport, cached assets, network	Reproduce in current Chrome; preserve exact manuscript size/cast facts
Job appears stuck after restart	Job state and startup reconciliation	Allow reconciliation; retry only from the visible action
Archive import fails	Preflight reason, version, hash, free space	Keep original archive unchanged; fix the source or use matching version
Database reports wrong family	Exact filename and identity evidence	Restore known matching pair; ask for review before any conversion

Chronicle-specific diagnosis

Memory is per canonical page revision. A heavier manuscript and cast can produce different results on another device even with the same model because the relevant prompt is larger. Ink Morrow prioritizes the Main Character, then support cast, then background setting and background cast. If the story is Main Character driven, that perspective anchor is always included whether or not the page names the Main Character.

A clickable failure records the error code, reason, model, and route to repair. Information entropy can make repair difficult, but repeated failure is not proof that the manuscript is corrupt. Diagnose the exact page, evidence, prompt pressure, and model response.

12 Recovery decision tree

1. **Is current data readable?** If yes, export a portable backup before changing anything.
2. **Is this a configuration or provider failure?** If yes, fix configuration; do not restore data.
3. **Is only one job/revision affected?** Use the product's repair, retry, undo, or recovery path.
4. **Was canon truncated?** Use immediate undo or Chronicle recovery while the surviving-head fingerprint matches.
5. **Did an update/storage operation fail?** Stop the app and restore the complete pre-change cold copy.
6. **Is the latest backup uncertain?** Restore into a separate directory and compare; never experiment on the only copy.

DEFINITION OF RECOVERED

The application starts without refusal, catalogue counts and hierarchy are correct, active prose and revisions agree, Chronicle coverage is understood, Codex corrections remain, media resolves, a fresh archive validates, and normal work has not resumed until these checks pass.

13 Command reference

```
# start
npm start

# full local verification (does not run browser E2E)
npm test

# explicit browser suite
npm run test:e2e

# brand residue guard
npm run check:brand

# record exact code
git rev-parse HEAD

# record Node
node --version
```

Do not run repair SQL found on the Internet. Do not use destructive Git commands against the data directory. Do not test with a real provider key unless the test is explicitly paid and bounded.

14 Operator checklists

Before writing

- Correct installation and data directory.
- Recent backup exists.
- Provider role/model and spending limit are intentional.
- No unresolved startup error.
- Correct manuscript and cast are visible.

Before maintenance

- Active work stopped.
- Full Gate archive downloaded.
- Complete cold data copy made.
- Current commit/configuration recorded.
- Rollback build available.

Before declaring success

- Startup, unlock, Library, Desk, Chronicle, Codex, Gallery, and Gate checked.
- Representative export opened.
- Backup preflight succeeds.
- Logs contain no secrets or repeated unexpected failures.
- New state is documented.